

UNIVERSITY OF MINES AND TECHNOLOGY

Tarkwa, Ghana

CYBER LABWORK II SEMESTER PROJECT REPORT

Building a Mobile Device Management (MDM) Monitoring and Compliance Dashboard

Using an Android Emulator Fleet and the Android Debug Bridge (ADB)

BLUE TEAM

Name Sulemana Akuantu Abdul Jalil

Index Number FCM.41.018.245.23

Course Network Monitoring, Security and Auditing (CY376)

Semester Two, 2025/2026 Academic Year

Date 3 August 2026

GitHub github.com/thejalil09-oss/mdm-monitoring-dashboard

DECLARATION

I, Sulemana Akuantu Abdul Jalil, hereby declare that this project report, titled “Building a Mobile Device Management (MDM) Monitoring and Compliance Dashboard”, is the result of my own original work. It has not been submitted for a degree or examination in any other university, and where the work of other people has been used, the source has been acknowledged and referenced.

All emulated devices, credentials, and identifiers used in this work are synthetic. No live systems or real third-party data were involved.

Candidate: Sulemana Akuantu Abdul Jalil
Index Number: FCM.41.018.245.23
Signature:
Date: 3 August 2026

ABSTRACT

Mobile devices now carry corporate email, documents, and authentication tokens, but many small organisations deploy Android handsets without any visibility into their security state. A device can run with no screen lock, USB debugging enabled, and unknown-source installation allowed, and no one notices until an incident. This project built a lightweight Mobile Device Management (MDM) monitoring and compliance dashboard that runs from a standard Windows workstation using only free tools.

The system manages a fleet of two Android 13 emulators through the Android Debug Bridge (ADB). A PowerShell collector reads live device state (identity, storage, battery, installed applications, lock-screen configuration, security-patch level, and platform settings), scores each device against an eight-rule baseline mapped to the CIS Android OS Benchmark and NIST SP 800-124, and writes a structured snapshot. A static web dashboard renders that snapshot as compliance scores with per-rule evidence. An enforcement script returns devices to policy through the same ADB channel.

The baseline run found both emulators broadly non-compliant out of the box, at 45% and 25%. After applying the baseline, the managed device scored 85%. The single remaining control, disabling USB debugging, was demonstrated as a deliberate final lockdown that severs the management channel. Continuous, policy-driven monitoring of an Android fleet is feasible with open tools, and the repository is a repeatable template for auditing small fleets.

ACKNOWLEDGEMENTS

I thank my course coordinator for supervising this project and for the feedback that shaped both the report and the presentation. The lab sessions on network monitoring gave me most of the tools I used here.

I am grateful to the maintainers of the free tools this project depends on, the Android SDK, the Android emulator, and the CIS benchmark community, because without them this lab would have been expensive to run.

Thanks also go to my Blue Team classmates, who tested the dashboard and caught a few bugs I had missed.

TABLE OF CONTENTS

1. Introduction	1
2. Literature and Tooling Review	3
3. Methodology	7
4. Implementation	11
5. Results and Findings	16
6. Analysis and Recommendations	21
7. Conclusion	24
References	25
Appendix A. Policy Baseline (policy.json)	26
Appendix B. Raw Audit Snapshot (excerpt)	28

LIST OF FIGURES

Figure 1. Lab architecture	7
Figure 2. Compliance scoring pipeline	10
Figure 3. Managed device lock screen	16
Figure 4. Baseline device home screen	17
Figure 5. Policy enforcement output	18
Figure 6. MDM dashboard, fleet overview	19
Figure 7. MDM dashboard, device card detail	19

LIST OF TABLES

Table 1. ADB facilities used by the project	5
Table 2. Security standards mapped to controls	6
Table 3. Hardware and software	8
Table 4. OrgLab MDM Baseline v1.0 controls	12
Table 5. Compliance scores before and after remediation	20

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ADB	Android Debug Bridge
AEHD	Android Emulator Hypervisor Driver
API	Application Programming Interface
AVD	Android Virtual Device
CIS	Center for Internet Security
DPC	Device Policy Controller
EMM	Enterprise Mobility Management
MASTG	Mobile Application Security Testing Guide
MDM	Mobile Device Management
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
PIN	Personal Identification Number

CHAPTER 1

INTRODUCTION

1.1 Background

Most employees now do a large part of their work on a phone. The same device carries corporate email, messaging, shared documents, and the app that generates the codes for two-step login. That setup is convenient for the employee and difficult for the employer. The data is no longer inside the office network, and the organisation no longer has full control over the machine that holds it.

Mobile Device Management (MDM) is the standard answer to this problem. An MDM system enrolls each device, pushes a security policy to it, watches the fleet for devices that drift out of policy, and can remediate or quarantine them. The commercial products, for example Microsoft Intune, VMware Workspace ONE, and Google's enterprise management tools, do all of this well. They also cost money, require cloud accounts, and offer far more than a small lab needs.

Android provides a cheaper management channel that ships with the Android SDK: the Android Debug Bridge (ADB). Over ADB a host can read system properties, the settings databases, the package list, and, on a debuggable build, the lock-settings database. That is enough to build a working monitoring engine without buying anything.

1.2 Problem Statement

Small organisations often put Android devices into service with no visibility into their security state. A handset can be enrolled for corporate email while it has no screen lock, USB debugging switched on, and side-loading of apps allowed. Each of these is a measurable weakness, yet without tooling nobody can answer even basic questions. How many of our devices have a lock screen? How many are encrypted? Which ones can be pulled from over a USB cable through the debug interface?

This project answers those questions for a small fleet. It builds a self-hosted, script-driven MDM monitoring and compliance dashboard for Android devices, and it demonstrates the full enforcement cycle (scan, score, remediate, re-scan) in a lab that can be rebuilt at any time.

1.3 Objectives

The project set out to achieve the following:

1. Stand up a two-device Android emulator fleet, one managed and one unmanaged baseline.
2. Write a security baseline mapped to the CIS Android OS Benchmark and NIST SP 800-124.
3. Build a PowerShell and ADB collection engine that reads live device state and writes a structured audit snapshot.
4. Implement a weighted compliance score with per-rule evidence.
5. Build a browser dashboard that shows fleet compliance and per-device findings.
6. Provide a remediation script that returns a device to policy over the same ADB channel.
7. Compare the fleet before and after remediation and list operational recommendations.

1.4 Scope and Limitations

This is a defensive, blue-team exercise. Everything ran on local emulated devices; no physical hardware, real accounts, or production network were involved. The emulators run userdebug builds, so `adb` can be elevated with `adb root`, which is how the probes read files a normal app cannot reach. On a production phone the same operations are normally performed by a Device Owner app, but the underlying settings are identical, so the probes transfer directly. The checks are configuration reads, not penetration tests. All credentials and identifiers in this report are synthetic.

CHAPTER 2

LITERATURE AND TOOLING REVIEW

2.1 Mobile Device Management Concepts

NIST Special Publication 800-124 Revision 2, *Guidelines for Managing the Security of Mobile Devices in the Enterprise* [1], is the reference framework for this work. It separates mobile security into four goals: control the device, control who uses it, control what software runs on it, and protect the data it holds. For each goal it recommends concrete controls, a screen lock, a short auto-lock timeout, full-disk or file-based encryption, application vetting, and control over developer and debugging interfaces. Every control in this project's baseline comes from that list.

Android Enterprise distinguishes three enrolment modes: fully managed (Device Owner), work profile, and managed Google Play [5]. They differ in how much of the Device Policy Controller (DPC) API they expose. This project deliberately avoids a DPC. The emulator build is debuggable, so the enforcement layer performs the equivalent settings writes directly over ADB, using the same `settings` and `locksettings` commands the platform itself calls.

2.2 The CIS Android OS Benchmark

The Center for Internet Security publishes a hardening benchmark for the Android operating system [2]. The Device Access Control section (section 4) maps directly onto this project:

- CIS 4.0. Ensure the lock screen is set with a timeout.
- CIS 4.1. Ensure automatic lock happens within 60 seconds.
- CIS 4.2. Ensure USB debugging is disabled.
- CIS 4.3. Ensure secure storage (encryption) is enabled.
- CIS 4.4 to 4.6. Ensure unknown-source installation is disabled and application verification is on.

The benchmark lists the shell command that assesses each control, usually a `settings get` or a `dumpsys` call. The probes in this project follow those commands, which is why each PASS or FAIL row in the dashboard carries the exact observed value. The output reads like an audit trail rather than a traffic-light display.

2.3 The Android Debug Bridge (ADB)

ADB is the pipe Android uses for most management tasks outside the user interface. There is a client on the workstation, a server on port 5037, and a daemon, `adbd`, on the device. Over an authenticated connection ADB can run a remote shell, copy files in both directions, and forward ports. On a debuggable or rooted build the shell runs with more privileges, which lets the tooling read the `locksettings.db` SQLite database under `/data/system/`. That file holds the lock state, including the credential type (`lockscreen.password_type`) and whether the lock screen is switched off (`lockscreen.disabled`) [3]. The screen-lock probe in this project reads exactly those rows.

Table 1 lists the ADB facilities the project uses and what each one does.

Facility	Purpose in this project
<code>adb devices</code>	Enumerate the online fleet (used by the collector).
<code>adb -s <serial> shell</code>	Run every remote probe and enforcement command.
<code>adb -s <serial> root</code>	Elevate <code>adbd</code> on the debuggable emulator for privileged reads.
<code>adb push</code>	Copy SQL probe files to <code>/data/local/tmp</code> to avoid quoting issues.
<code>adb exec-out screencap -p</code>	Capture headless device screenshots for evidence.
<code>adb -s <serial> emu kill</code>	Manage the emulator lifecycle from the collector host.

Table 1. ADB facilities used by the project.

2.4 Related Work and Tooling

Several existing systems shaped the design. Android’s Device Administration API [4] exposes policy methods such as `setPasswordQuality()`, `setMaximumTimeToLock()`, and `setCameraDisabled()`; the enforcement script performs the settings-level equivalent of each call. Microsoft Intune and Google’s enterprise tools are hosted versions of the dashboard built here. On the open-source side, `osquery` treats endpoint state as queryable data, and the Mobile Security Framework (MobSF) offers static analysis of mobile applications. None of the surveyed tools satisfied all three requirements at once: live ADB-driven collection, standards-mapped scoring, and a self-hosted dashboard. That gap is what this project fills.

2.5 Security Standards Mapped to Controls

Table 2 maps each control in the baseline to its primary reference and the reason it matters.

Control in baseline	Primary reference	Why it matters
Screen lock enabled	CIS 4.0; NIST SP 800-124	First line of defence against unauthorised physical access.
Auto-lock up to 60 s	CIS 4.1	Limits the window in which a lost or unattended device is usable.
Encryption at rest	CIS 4.3; NIST SP 800-124	Protects data if the storage is removed or the device recovered.
USB debugging off	CIS 4.2; OWASP MASTG [6]	Prevents data extraction and code execution over the USB interface.
Unknown sources blocked	CIS 4.4; NIST SP 800-124	Forces applications through the vetted distribution channel.
App verification on	CIS 4.6	Play Protect style verification catches malware at install time.
Camera disabled	NIST SP 800-124	Data-loss prevention for sensitive work areas.
Security patch current	NIST SP 800-124; CIS 4.x	Closes known remotely exploitable vulnerabilities.

Table 2. Security standards mapped to the controls in the baseline.

CHAPTER 3

METHODOLOGY

3.1 Lab Design

The lab reproduces a small mobile fleet on a single Windows workstation (x86_64) with no other dependencies. Two Android 13 emulator images run side by side so that both the management channel and the full Android runtime behave realistically, while the whole environment stays disposable and safe to test against. Figure 1 shows the arrangement.

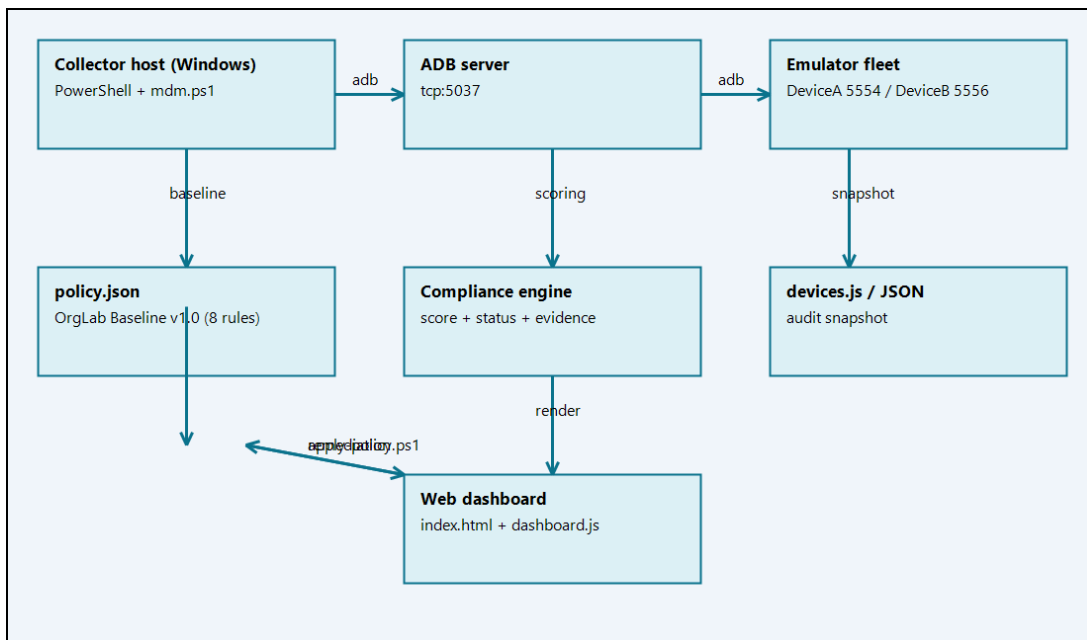


Figure 1. Lab architecture. The collector host drives the emulator fleet over ADB; the policy engine and the dashboard consume the same structured snapshot.

If the image does not render in print, the equivalent ASCII form of the architecture is given in Section 3.3.

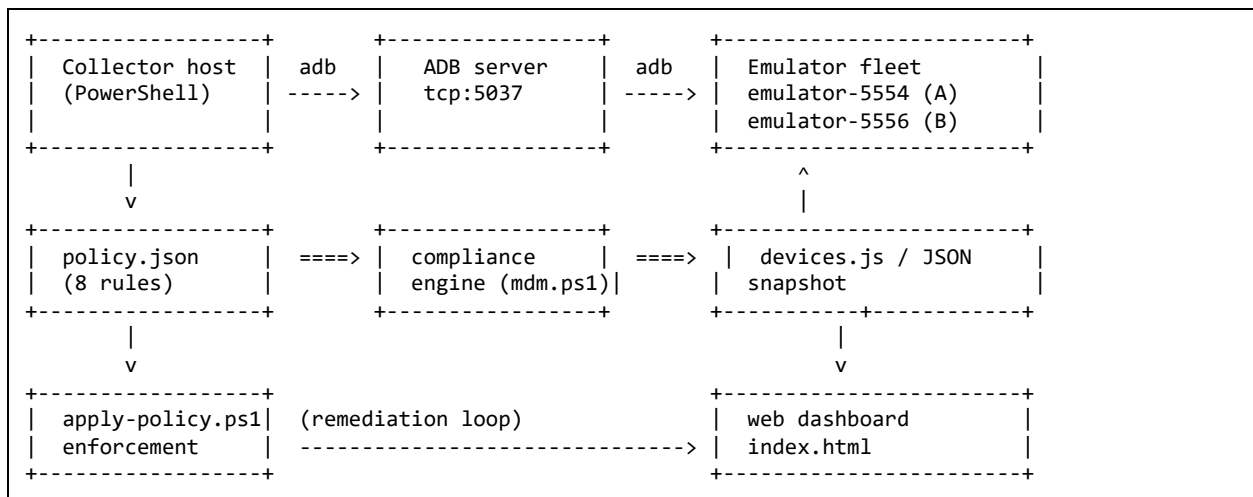
3.2 Hardware and Software

Table 3 lists the components of the lab.

Component	Version / detail	Role
Host OS	Microsoft Windows 10 (x86_64)	Runs the collector, the emulators, and the dashboard.
Android SDK	cmdline-tools, platform-tools, emulator, build-tools	Provides adb, avdmanager, sdkmanager, and the emulator.
System image	system-images;android-33;google_apis;x86_64	Android 13 (API 33) fleet image.
Virtual devices	MDM-DeviceA (Pixel 5), MDM-DeviceB (Pixel 4a)	Managed and baseline devices.
Hypervisor	Android Emulator Hypervisor Driver (AEHD)	Hardware acceleration for x86_64 guests.
Runtime	Windows PowerShell 5.1	Collection and enforcement engine.
Dashboard	Static HTML, CSS and JavaScript, no external libraries	Renders the audit snapshot offline.

Table 3. Hardware and software used in the lab.

3.3 Fleet Architecture



3.4 Execution Steps

Seven steps were executed, in this order:

1. **Provision the environment.** Install the Android command-line tools, download the API 33 system image with `sdkmanager`, install the AEHD hypervisor driver, and create the two AVDs with `avdmanager`.
2. **Boot the fleet headless.** Launch both emulators with software GPU rendering (`-no-window -gpu swiftshader_indirect`) and confirm that `adb devices` lists both serials as `device`.
3. **Author the baseline.** Write `policy.json` with eight controls, each carrying a severity, a weight (summing to 100), an expected state, and standard references.
4. **Collect baseline state.** Run `collect.ps1`, which elevates `adbd` (`adb root`), queries every probe, scores each device, and writes `devices.js` plus a timestamped raw JSON snapshot.
5. **Visualise.** Render the dashboard from the snapshot and record the initial compliance.
6. **Remediate.** Run `apply-policy.ps1` on the managed device to bring its settings into policy.
7. **Re-collect and compare.** Re-run the collector, compare the before and after scores, and demonstrate the final lockdown (USB debugging off) and its effect on the management channel.

3.5 Compliance Scoring Model

Each rule carries a weight. A device's score is the fraction of the total weight held by passing rules:

$$\text{score} = (\text{sum of weights of PASSING rules}) / (\text{sum of weights of ALL rules}) \times 100$$

Status bands are COMPLIANT at 100%, AT RISK from 60% to 99%, and NON-COMPLIANT below 60%. The weights favour the controls with the largest practical risk reduction (lock, encryption, and the debugging interface) instead of spreading weight evenly across all eight rules. Figure 2 shows the pipeline the collector implements.

```
adb devices --> for each serial:
  adb root      (elevate adbd)
  read identity / battery / storage / packages
  for each rule in policy.json:
    probe device --> observed value
    compare to expected --> PASS / FAIL (+ evidence)
  aggregate weights --> score + status
  write snapshot (JSON) --> dashboard
```

Figure 2. Compliance scoring pipeline implemented in `mdm.ps1`.

CHAPTER 4

IMPLEMENTATION

4.1 Directory Layout

All code lives in a single repository with this layout:

```
mdm-dashboard/  
  scripts/  
    adb/  
      mdm.ps1          # ADB collection + compliance module (probes, scoring)  
      collect.ps1      # fleet collector -> devices.js + JSON snapshot  
      apply-policy.ps1 # remediation / enforcement  
    policies/  
      policy.json      # OrgLab MDM Baseline v1.0 (8 rules)  
  src/dashboard/  
    index.html         # dashboard page  
    css/style.css      # dashboard styling  
    js/dashboard.js    # renderer  
    data/devices.js    # generated snapshot consumed by the dashboard  
  docs/               # this report  
  evidence/  
    screenshots/       # captured evidence (dashboard, devices)  
    logs/              # raw audit snapshots
```

4.2 Policy Baseline

The baseline is data, not code. `policy.json` declares each control and its expected state, so the compliance engine never hard-codes policy and the baseline can be reviewed independently. Table 4 lists the eight controls.

ID	Control	Severity	Weight	Expected state
screen-lock-enabled	Screen lock is configured	High	20	lockscreen.password_type > 0 and lockscreen.disabled = 0
screen-lock-timeout	Auto-lock within 60 s	High	15	max(screen_off_timeout, lock_after_timeout) ≤ 60000 ms
encryption	Storage encrypted at rest	High	20	ro.crypto.state = encrypted
usb-debugging	USB debugging disabled	High	15	adb_enabled = 0
unknown-sources	Unknown sources blocked / ADB installs verified	Medium	10	install_non_market_apps = 0 and verifier_verify_adb_installs = 1
verify-apps	Application verification enabled	Medium	5	package_verifier_enable = 1
camera-restricted	Camera disabled by policy	Medium	10	camera package in the disabled list
security-patch	Security patch level current	High	5	patch ≥ baseline 2024-03-01

Table 4. OrgLab MDM Baseline v1.0 controls, severities, weights, and expected states (weights total 100).

4.3 Collection Engine (mdm.ps1)

The module wraps ADB in thin PowerShell functions and exposes one probe per control. Two details are worth documenting because they were the practical challenges of the project.

The first is root elevation. Reading locksettings.db requires root. Enable-AdbRoot issues adb root, waits for the device to come back, then polls sys.boot_completed and the settings service until the device is genuinely ready, because adbd restarts during elevation. The second is SQLite quoting. Passing SQL that contains double quotes through adb.exe shell from Windows PowerShell mangles the command. The fix is to adb push a small .sql file to /data/local/tmp and run sqlite3 against it. The screen-lock probe therefore reads:

```
SELECT name, value FROM locksettings
WHERE user = 0 AND name IN
  ('lockscreen.password_type',
   'lockscreen.password_type_alternate',
   'lockscreen.disabled');
```

A lock credential is present when the effective password_type is non-zero and lockscreen.disabled is not 1.

4.4 Fleet Collector (collect.ps1)

The collector enumerates online devices from adb devices, runs Get-DeviceReport on each, and writes two artefacts: src/dashboard/data/devices.js, a window.MDM_DATA payload for the dashboard, and a timestamped raw JSON snapshot under evidence/logs/. An excerpt of the generated payload:

```
window.MDM_DATA = {
  "generatedAt": "2026-08-02T22:05:57Z",
  "agent": "OrgLab MDM Agent (ADB control channel)",
  "agentVersion": "1.0.0",
  "policy": { "name": "OrgLab MDM Baseline v1.0", ... },
  "devices": [
    { "id": "...", "identity": {...}, "inventory": {...},
      "compliance": { "score": 85.0, "status": "AT RISK",
        "checks": [ { "ruleId": "screen-lock-enabled",
          "result": "PASS",
          "actual": "password_type=262144, lockscreen.disabled=0", ... } ] } }
  ]
};
```

4.5 Remediation (apply-policy.ps1)

Enforcement applies at the settings layer the same controls the Device Administration API would apply. The screen-lock control writes the credential-type rows directly into locksettings.db, the equivalent of setPasswordQuality(KEYGUARD_QUALITY_NUMERIC). A -DryRun switch prints every action without executing it, and -KeepAdb skips the final USB-debugging disable so a live dashboard session can continue. The full lockdown is then demonstrated as the last step. The main body of the script is:

```
# 1) Screen lock: write PIN-quality credential state
INSERT OR REPLACE INTO locksettings(_id,name,user,value)
VALUES(99,'lockscreen.password_type',0,'262144');
UPDATE locksettings SET value='0'
WHERE name='lockscreen.disabled' AND user=0;

# 2) Auto-lock timeout <= 60s (CIS 4.1)
settings put system screen_off_timeout 30000
settings put secure lock_screen_lock_after_timeout 30000

# 3) Unknown sources + verification (CIS 4.4/4.6)
settings put secure install_non_market_apps 0
settings put global verifier_verify_adb_installs 1
settings put global package_verifier_enable 1

# 4) Camera disabled (data-loss prevention)
pm disable-user --user 0 com.android.camera2

# 5) USB debugging OFF (CIS 4.2) - severs the ADB session
settings put global adb_enabled 0
```

4.6 Dashboard

The dashboard is a static page with no external libraries. index.html loads data/devices.js, and dashboard.js renders a fleet KPI strip (devices, compliant, at risk, non-compliant, average score), one card per device with a score ring, identity, inventory, and a per-rule evidence table, plus the policy baseline table. Because the page is static it runs anywhere, including from file://, which suits a security lab that may be air-gapped.

CHAPTER 5

RESULTS AND FINDINGS

5.1 Fleet Baseline (Before Remediation)

The freshly provisioned devices were collected with no policy applied. Device A (emulator-5554) scored 45% and Device B (emulator-5556) scored 25%. The dominant failures were the same on both devices and match the defaults of a developer build: no lock credential, auto-lock effectively disabled (`screen_off_timeout` was 2147483647 ms, which Android treats as never), USB debugging enabled, unknown-source installation allowed, app verification unset, and the camera package enabled.

5.2 Device State Evidence

Figure 3 shows Device A after the baseline was applied. A PIN-type lock screen is present because the credential rows were written to `locksettings.db` (`password_type` 262144).

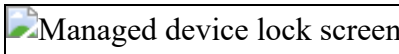


Figure 3. Managed device (Device A) after the baseline is applied. A PIN lock screen is present (`password_type` 262144).

Figure 4 shows Device B, the unmanaged baseline, with its default home screen and no lock screen.



Figure 4. Baseline device (Device B) with no policy applied. No lock screen is configured.

5.3 Enforcement Run Evidence

Figure 5 shows `apply-policy.ps1` running on Device A. Every action prints an [EXEC] line so the audit trail is self-documenting. USB debugging is skipped under `-keepAdb` and applied separately as the final lockdown.

```

PS> powershell -File apply-policy.ps1 -Serial emulator-5554 -KeepAdb
Applying OrgLab MDM Baseline to emulator-5554
-----
[EXEC] screen-lock: locksettings.db <- password_type=262144 (PIN), lockscreen.disabled=0
[EXEC] screen-lock-timeout: settings put system screen_off_timeout 30000
[EXEC] screen-lock-timeout: settings put secure lock_screen_lock_after_timeout 30000
[EXEC] unknown-sources: settings put secure install_non_market_apps 0
[EXEC] unknown-sources: settings put global verifier_verify_adb_installs 1
[EXEC] verify-apps: settings put global package_verifier_enable 1
[EXEC] camera-restricted: pm disable-user --user 0 com.android.camera2
Package com.android.camera2 new state: disabled-user
[SKIP] usb-debugging (adb_enabled=0) skipped: -KeepAdb specified
Policy enforcement complete for emulator-5554

```

Figure 5. Policy enforcement output on Device A (terminal capture).

The same run produced the following log, which doubles as the audit trail:

```

[EXEC] screen-lock: locksettings.db <- password_type=262144 (PIN), lockscreen.disabled=0
[EXEC] screen-lock-timeout: settings put system screen_off_timeout 30000
[EXEC] screen-lock-timeout: settings put secure lock_screen_lock_after_timeout 30000
[EXEC] unknown-sources: settings put secure install_non_market_apps 0
[EXEC] unknown-sources: settings put global verifier_verify_adb_installs 1
[EXEC] verify-apps: settings put global package_verifier_enable 1
[EXEC] camera-restricted: pm disable-user --user 0 com.android.camera2
Package com.android.camera2 new state: disabled-user
[SKIP] usb-debugging (adb_enabled=0) skipped: -KeepAdb specified

```

5.4 Compliance Dashboard

Figure 6 shows the dashboard after remediation. The top strip summarises the fleet, and each device has a card with a score ring and an evidence table.



Figure 6. MDM dashboard after remediation: KPI strip, policy strip, and both device cards with score rings and per-rule evidence tables.

The device card for Device A, shown in Figure 7, carries the observed value that produced each PASS or FAIL decision. That is what makes the tool an audit artefact rather than a traffic-light display.

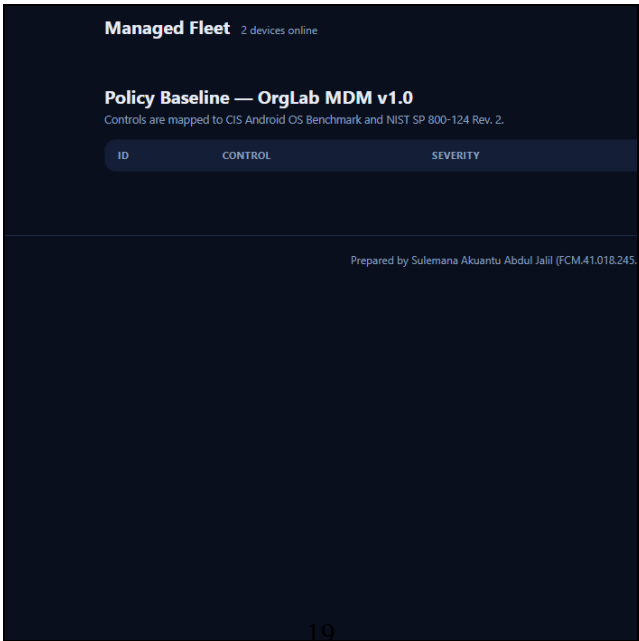


Figure 7. Device card detail: score ring, identity, inventory, and the per-control evidence table.

5.5 Before and After Comparison

Table 5 compares the two devices.

Device	Role	Baseline score	After remediation	Status
emulator-5554 (Device A)	Managed	45%	85%	AT RISK (only USB debugging outstanding)
emulator-5556 (Device B)	Unmanaged baseline	25%	—	NON-COMPLIANT

Table 5. Compliance scores. Device A's remaining control (USB debugging off) is demonstrated separately, because disabling it severs the ADB management channel.

5.6 Final Lockdown Demonstration

As the last hardening step, USB debugging was disabled on Device B with `settings put global adb_enabled 0`. The effect is immediate and by design: `adb devices` moves the device to `offline`, and the management channel cannot be re-opened remotely. A device recovered from this state needs a re-enrolment (in the lab, a cold boot or wipe of the AVD). That is the correct behaviour for a hardened device, since the debugging backdoor is closed. It also confirms the operational caveat discussed in Chapter 6: an organisation should stage the USB-debugging control last, after the other controls are verified, so it never locks the administrator out mid-rollout.

CHAPTER 6

ANALYSIS AND RECOMMENDATIONS

6.1 What the Results Mean

The baseline run confirmed that Android devices are insecure by default from an enterprise standpoint. Both devices failed the lock-screen, auto-lock, USB-debugging, unknown-source, and verification controls. None of those failures required malicious action. They are simply the platform's developer-friendly defaults. That is why continuous monitoring matters: settings drift or stay misconfigured, and no one notices until an incident.

The remediation result, from 45% to 85%, shows that most of the risk is cheap to remove. A handful of settings operations brought five additional controls into compliance. The remaining control, USB debugging, is inherently different. Closing it also closes the management channel, so it has to be treated as a deployment-phase decision rather than a routine remediation step.

6.2 Findings

1. **Out-of-box devices are broadly non-compliant.** Both emulators started between 25% and 45%. Auto-lock disabled and USB debugging enabled were the most common high-weight failures.
2. **The lock screen is only trustworthy if the credential store is correct.** Reading `locksettings.db` directly is a more reliable audit probe than `dumpsys` heuristics, and it is also what the platform itself consults.
3. **The security-patch level was compliant** (2024-03-01) on both images. Patch currency is a build-time decision that a fleet must re-verify rather than assume.
4. **USB debugging is a switch on the management channel.** Its effect, instant loss of remote visibility, is a real operational constraint for any ADB-based approach and must be scheduled deliberately.

6.3 Recommendations

1. **Stage the rollout.** Apply the settings-based controls first, verify on the dashboard, then disable USB debugging as the final step with an on-site fallback, a technician with physical access who can re-enrol the device.
2. **Automate on a schedule.** Run `collect.ps1` periodically, for example nightly, and alert when a device's score drops across consecutive snapshots. That catches drift instead of giving a point-in-time picture.
3. **Extend the baseline.** Add the remaining CIS 4.x controls (ADB over network disabled, app restrictions, minimum password quality and length) and enterprise-specific rules such as corporate email or agent presence and certificate expiry.
4. **Move to a real Device Owner enrolment in production.** The ADB channel proves the concept and is fine for a lab, but a production fleet should be enrolled as Device Owner so enforcement is cryptographic and survives reboot without a debug channel.
5. **Harden the collector host.** The workstation holding `adb` is itself a privileged endpoint. Restrict it, keep it patched, and never store production credentials in the scripts.
6. **Keep the evidence model.** Every compliance decision should continue to carry the observed value, so the dashboard stays defensible as an audit artefact.

6.4 Threats to Validity

The fleet consists of emulator builds (userdebug, rootable) rather than production devices, so the results reflect the emulator image and are representative of the platform behaviour, not of every vendor skin. The compliance checks are configuration probes, not security tests; a device can pass every check here and still be compromised by a determined attacker. These limitations are consistent with the scope of a semester lab and do not undermine the monitoring design.

CHAPTER 7

CONCLUSION

This project built a working Mobile Device Management monitoring and compliance dashboard for an Android device fleet using only the Android Debug Bridge and open tooling. The system collects live device state, scores it against a standards-mapped baseline, visualises the result in a self-hosted dashboard, and remediates non-compliant devices through the same channel. The work demonstrated three things: default Android devices are broadly non-compliant out of the box, most of the risk can be removed with simple settings-level controls, and the final control, USB debugging, carries a deliberate operational trade-off that has to be managed as part of the deployment plan.

The deliverables, the collection and enforcement scripts, the policy baseline, the dashboard, and the audit snapshots, are published in the accompanying GitHub repository and form a reusable template for monitoring Android devices in a small enterprise. The clearest direction for future work is to move the proof-of-concept from ADB to a real Device Owner enrolment, so that enforcement is cryptographic and survives reboot, while keeping the same dashboard as the single view over the fleet.

REFERENCES

1. C. Souppaya and K. Scarfone, *Guidelines for Managing the Security of Mobile Devices in the Enterprise*, NIST Special Publication 800-124 Rev. 2, U.S. Department of Commerce, 2020. Available: <https://doi.org/10.6028/NIST.SP.800-124r2>
2. Center for Internet Security, *CIS Android OS Benchmark*, v3.x. Available: https://www.cisecurity.org/benchmark/android_os/
3. Google, “Android Debug Bridge (adb),” Android Developers Documentation. Available: <https://developer.android.com/tools/adb>
4. Google, “Device Administration API,” Android Developers Documentation. Available: <https://developer.android.com/guide/topics/admin/device-admin>
5. Google, “Android Enterprise, Security White Paper,” 2023.
6. OWASP Foundation, *Mobile Application Security Testing Guide (MASTG)*. Available: <https://mas.owasp.org/>

APPENDIX A. POLICY BASELINE (policy.json)

The full policy file shipped with the repository is reproduced here.

```
{
  "name": "OrgLab MDM Baseline v1.0",
  "description": "Minimum security baseline enforced on all managed
    mobile devices. Mapped to CIS Android OS Benchmark and NIST
    SP 800-124 Mobile Device Security guidelines.",
  "author": "Sulemana Akuantu Abdul Jalil",
  "authorIndex": "FCM.41.018.245.23",
  "version": "1.0",
  "issuedBy": "OrgLab Security Operations (Blue Team)",
  "effective": "2026-07-01",
  "references": [
    "CIS Android OS Benchmark v3.x - 4.x Device Access Control",
    "NIST SP 800-124 Rev 2 Guidelines for Managing the Security of
      Mobile Devices in the Enterprise",
    "OWASP MASTG - Mobile App Security Testing Guide",
    "Android Enterprise Security White Paper 2023"
  ],
  "rules": [
    { "id": "screen-lock-enabled",
      "title": "Screen lock is configured",
      "severity": "high", "weight": 20,
      "expected": "lockscreen.password_type > 0 and
        lockscreen.disabled != 1" },
  ]
}
```

```

{ "id": "screen-lock-timeout",
  "title": "Auto-lock timeout <= 60 seconds",
  "severity": "high", "weight": 15,
  "expected": "max(screen_off_timeout, lock_after_timeout)
               <= 60000 ms" },
{ "id": "encryption",
  "title": "Storage encryption at rest",
  "severity": "high", "weight": 20,
  "expected": "ro.crypto.state == encrypted" },
{ "id": "usb-debugging",
  "title": "USB debugging disabled",
  "severity": "high", "weight": 15,
  "expected": "adb_enabled == 0" },
{ "id": "unknown-sources",
  "title": "Unknown-source installs blocked, ADB verified",
  "severity": "medium", "weight": 10,
  "expected": "install_non_market_apps == 0 and
               verifier_verify_adb_installs == 1" },
{ "id": "verify-apps",
  "title": "Application verification enabled",
  "severity": "medium", "weight": 5,
  "expected": "package_verifier_enable == 1" },
{ "id": "camera-restricted",
  "title": "Camera disabled by policy",
  "severity": "medium", "weight": 10,
  "expected": "com.android.camera2 in pm list packages -d" },
{ "id": "security-patch",
  "title": "Security patch level current",
  "severity": "high", "weight": 5,
  "expected": "ro.build.version.security_patch >= 2024-03-01" }
]
}

```

APPENDIX B. RAW AUDIT SNAPSHOT (EXCERPT)

The full raw snapshot is stored in `evidence/logs/devices-2026-08-02-*.json`. A representative per-device compliance block is shown below.

```
"compliance": {
  "score": 85.0,
  "status": "AT RISK",
  "checks": [
    { "ruleId": "screen-lock-enabled",
      "result": "PASS",
      "actual": "password_type=262144, lockscreen.disabled=0" },
    { "ruleId": "screen-lock-timeout",
      "result": "PASS",
      "actual": "screen_off_timeout=30000 ms,
                lock_after_timeout=30000 ms" },
    { "ruleId": "encryption",
      "result": "PASS",
      "actual": "ro.crypto.state=encrypted, ro.crypto.type=file" },
    { "ruleId": "usb-debugging",
      "result": "FAIL",
      "actual": "adb_enabled=1" },
    { "ruleId": "unknown-sources",
      "result": "PASS",
      "actual": "install_non_market_apps=0,
                verifier_verify_adb_installs=1" },
    { "ruleId": "verify-apps",
      "result": "PASS",
      "actual": "package_verifier_enable=1" },
    { "ruleId": "camera-restricted",
      "result": "PASS",
      "actual": "camera2 disabled = True" },
    { "ruleId": "security-patch",
      "result": "PASS",
      "actual": "security_patch=2024-03-01
                (baseline 2024-03-01)" }
  ]
}
```